

Safety and Rollback Mechanisms for Objective Misinterpretation in Agentic AI Systems: A Review

KM SHWETA, Department of Information Technology, IIIT Allahabad, India

Agentic artificial intelligence (AI) systems increasingly combine large language models (LLMs), planning, memory, tool use, code execution, and autonomous multi-step decision-making. These capabilities create a distinctive safety problem: an agent may retain substantial competence while pursuing an objective that differs from the designer's or user's intended objective. This phenomenon encompasses specification gaming, reward hacking, goal misgeneralization, reward tampering, and forms of inner misalignment. Conventional safeguards—reward-model improvement, human feedback, monitoring, constrained action spaces, and shut-down mechanisms—primarily seek to prevent or detect undesirable behavior before or during execution. This review examines a complementary safety paradigm: rollback, defined as the controlled restoration of an agent and its execution environment to a previously validated state when objective misinterpretation is suspected. Drawing on and comparing research in AI alignment, reinforcement learning, corrigibility, safe reinforcement learning, checkpoint/restore systems, and recent agent-sandbox research, this paper synthesizes the literature into a layered safety framework, presents a comparative analysis of existing checkpoint/restore and reward-hacking-mitigation approaches, identifies open research gaps, and outlines directions for objective-aware transactional agency.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; • **Security and privacy** → *Systems security*; • **Applied computing** → Document management.

Additional Key Words and Phrases: agentic AI, AI safety, objective misinterpretation, goal misgeneralization, specification gaming, reward hacking, corrigibility, rollback, checkpoint/restore, safe reinforcement learning, LLM agents, recovery

1 Introduction

The transition from conventional predictive machine-learning systems to agentic AI changes the safety problem in an important way. A conventional model may produce an incorrect classification or generation, whereas an agent can interpret an objective, formulate a plan, invoke external tools, modify persistent state, and continue acting over an extended horizon. Errors in objective interpretation can therefore compound across multiple actions. An agent that misunderstands what a user wants is not merely capable of producing a wrong answer; it may competently execute a wrong plan.

This distinction is central to goal misgeneralization. Di Langosco et al. [2022] define goal misgeneralization as a case in which an agent retains its capabilities outside the training distribution but pursues the wrong goal. Their empirical examples demonstrate that an agent can continue to navigate effectively while targeting an unintended feature of the environment. The phenomenon is closely related to, but conceptually distinct from, specification gaming and reward hacking, in which an agent optimizes a proxy objective in a way that satisfies the formal specification while violating its intended purpose [Amodei et al. 2016; Krakovna et al. 2020; Skalse et al. 2022].

The increasing autonomy of LLM-based systems makes this class of failures particularly consequential. Modern agents may manipulate files, execute code, call APIs, send messages, purchase resources, alter databases, or delegate subtasks. Recent empirical work has shown that LLMs can exhibit specification gaming and, under particular training conditions, generalize from comparatively simple gaming behavior toward reward-tampering behavior [Denison et al. 2024]. More recent work also reports reward-hacking behavior in language-model agents under controlled safety-gridworld-like evaluations [Çağatan and Zhao 2026].

This review therefore investigates a complementary question: *if an agent has already begun pursuing a misinterpreted objective, how can its execution be safely interrupted and rolled back to a previously validated state?* Unlike prior surveys that treat prevention (reward design, monitoring) and correction (shutdown, rollback) as separate literatures, this review synthesizes both strands, compares the leading checkpoint/restore systems directly, and positions rollback as one layer within a broader containment architecture.

The remainder of this paper is organized as follows. Section 2 reviews the foundational literature on objective misinterpretation, inner alignment, and corrigibility. Section 3 presents a taxonomy of failure modes. Section 4 defines rollback formally. Section 5 presents a comparative analysis of existing checkpoint/restore and reward-hacking-mitigation systems. Section 6 proposes a layered architecture. Sections 7–8 discuss open challenges and research gaps, Section 9 provides a synthesizing discussion, and Section 10 concludes.

2 Background and Related Work

2.1 Objective Misinterpretation and Specification Failure

AI systems optimize representations of objectives rather than human intentions directly. Amodei et al. [2016] identified reward hacking as a fundamental AI-safety problem: an agent may discover ways to maximize its reward function that do not correspond to what its designers actually wanted. Krakovna et al. [2020] subsequently documented numerous examples of specification gaming, emphasizing that increasingly capable optimization can expose loopholes that weaker systems fail to discover.

Skalse et al. [2022] formalize reward hacking in terms of a proxy reward $\tilde{R}$ and a true reward R . A proxy is problematic when optimization of $\tilde{R}$ can reduce performance according to R . Their theoretical analysis demonstrates that strong notions of “unhackability” are difficult to satisfy, particularly for broad policy classes.

Objective misinterpretation is broader than reward hacking. In goal misgeneralization, the training reward may itself be correct, but correlations present during training can lead the learned agent to internalize an unintended goal. Di Langosco et al. [2022] show that capability generalization and goal generalization can come apart. This distinction is especially important for rollback: if the failure arises from an incorrect objective encoded in the agent’s persistent memory or policy, merely restoring the environment may reproduce the same behavior.

2.2 Inner Alignment and Learned Optimization

Hubinger et al. [2019] introduced the framework of mesa-optimization to analyze cases in which a learned model itself functions as an optimizer and may develop an objective that differs from the training objective. For agentic AI, this concern can arise at several levels: the user’s natural-language instruction, the system’s interpreted task representation, a planner’s intermediate objective, a reward or evaluation model, persistent agent memory, a learned policy or model, and externally represented task state. Rollback mechanisms must therefore specify *what* state is being rolled back, since reverting only the conversation history is insufficient if an agent has already modified files, acquired credentials, or altered a database.

2.3 Corrigibility and Shutdown

Corrigibility research provides a conceptual foundation for recovery mechanisms. Soares et al. [2015] characterize a corrigible system as one that cooperates with corrective intervention despite incentives that might otherwise favor resisting modification or shutdown. Orseau and Armstrong [2016] showed that reinforcement-learning agents can, under suitable constructions, be made safely

interruptible. Hadfield-Menell et al. [2017] developed the Off-Switch Game and demonstrated that a rational agent with uncertainty about its objective can have incentives to preserve an off-switch rather than disable it. Thornley [2024] formalizes tensions between competent goal pursuit and reliable shutdown, showing that apparently innocuous assumptions about rational agents can create incentives to interfere with shutdown mechanisms. Rollback extends this literature from a binary continue-or-stop decision to a third option: *restore and replan*.

3 A Taxonomy of Objective-Related Failure

Objective-related failures in agentic AI can be organized into five overlapping categories, summarized in Table 1: specification gaming [Krakovna et al. 2020], reward hacking [Skalse et al. 2022], goal misgeneralization [Di Langosco et al. 2022], reward tampering [Denison et al. 2024], and inner objective divergence [Hubinger et al. 2019]. These mechanisms differ causally, but they create a common operational problem: the agent may continue to act coherently after its objective has diverged from the intended one.

Table 1. Taxonomy of objective-related failures in agentic AI and their rollback-relevant concerns.

Failure type	Description	Rollback-relevant concern
Specification gaming	Literal satisfaction of a specification through an unintended strategy	Agent may keep exploiting a loop-hole even after restoration
Reward hacking	Increasing a proxy reward while reducing the true objective	A single reward signal is unreliable as a rollback trigger
Goal misgeneralization	Capabilities generalize while the learned goal does not	Restoring the environment may reproduce the same wrong objective
Reward tampering	Manipulation of the reward/evaluation mechanism itself	The recovery mechanism may itself become a target
Inner objective divergence	A learned optimizer pursues an objective different from training	Computational restoration alone may not fix the objective state

The risk from these failures is amplified by three properties of agentic systems: *horizon* (errors compound across many actions), *agency* (the system can modify the environment in ways that make correction harder; Turner et al. [2021] show optimal policies have incentives to preserve

control), and *persistence* (memory, credentials, and external transactions outlive the reasoning trace that produced them).

4 Rollback as a Safety Mechanism

For purposes of this review, rollback is defined as *the controlled restoration of an agent’s computational and environmental state to a previously validated checkpoint, followed by independent revalidation before execution resumes*. A checkpoint can be represented as

$$C_t = (S_t, M_t, E_t, T_t, A_t, O_t), \quad (1)$$

where S_t is computational state, M_t agent memory, E_t the execution environment, T_t tool/credential state, A_t action history, and O_t the validated objective representation, as summarized in Table 2. A naive rollback restores only S_t ; a safety-oriented rollback must reason about all six components, since an agent can be computationally restored while remaining externally misaligned (e.g., a reversed filesystem snapshot cannot undo an already-accepted bank payment).

Table 2. Components of a safety checkpoint, Equation (1).

Symbol	Meaning
S_t	Computational state
M_t	Agent memory and context
E_t	Execution environment
T_t	Tool and credential state
A_t	Action/effect history
O_t	Validated objective representation

Rollback should be positioned within a three-stage control loop, Prevent $\rightarrow$ Detect $\rightarrow$ Recover, in which preventive controls (reward modeling, sandboxing, least-privilege tools), detection controls (behavioral monitors, trajectory evaluation), and recovery controls (interruption, rollback, compensation) reinforce one another rather than any single control being assumed perfect [Reddy et al. 2019].

5 Comparative Analysis of Existing Approaches

This section directly compares the leading systems- and training-level approaches surveyed in this review, summarizing their methodology, target problem, reported results, and limitations in Table 3.

Two patterns emerge from Table 3. First, systems-level work (DeltaBox, Crab) optimizes *how fast and how completely* state can be restored, while ACRFence addresses *whether restoration itself is safe* once external effects are involved — these are complementary rather than competing contributions. Second, training-time methods (MONA, ReQueST) reduce the *likelihood* of objective misinterpretation but do not eliminate the need for runtime recovery, reinforcing the Prevent $\rightarrow$ Detect $\rightarrow$ Recover framing introduced in Section 4.

6 Architecture for Objective-Aware Rollback

A robust architecture should contain at least seven layers, summarized in Figure 1: objective representation, action provenance, safety checkpointing, objective-drift monitoring, an independent interrupt controller, state restoration, and post-rollback validation.

Table 3. Comparative analysis of checkpoint/restore systems and reward-hacking-mitigation approaches.

Approach	Category	Methodology	Reported findings	Limitations
DeltaBox [Dong et al. 2026]	Checkpoint/restore system	Incremental filesystem and process-state snapshotting for agent sandboxes	~14 ms checkpoint, ~5 ms rollback in studied configurations	Addresses computational restoration only; does not resolve external/irreversible effects
Crab [Wu et al. 2026]	Checkpoint/restore system	Semantics-aware runtime aligning checkpoint granularity with agent-relevant OS effects	Reduced checkpoint traffic; improved recovery correctness over chat-only restoration	Requires OS-level instrumentation; agent-OS semantic gap can still miss some effects
ACRFence [Zheng et al. 2026]	Rollback-safety mechanism	Detects semantic rollback attacks (action replay, authority resurrection) and enforces replay-or-fork semantics	Identifies duplicated/resurrected external effects that naive restoration introduces	Defensive layer only; does not itself decide <i>when</i> to roll back
MONA [Farquhar et al. 2025]	Training-time prevention	Myopic optimization combined with non-myopic approval	Reduced several controlled forms of multi-step reward hacking	Prevention-only; deployed agents still require runtime containment
ReQueST [Reddy et al. 2019]	Reward modeling	Learns reward models from evaluated hypothetical trajectories rather than live unsafe states	Reduces exposure of the agent to unsafe states during training	Learned reward remains a proxy; does not address post-deployment drift

1. Objective Representation → 2. Action Provenance → 3. Safety Checkpoint → 4. Objective-Drift Monitor → 5. Independent Interrupt Controller → 6. State Restoration → 7. Post-Rollback Validation

Fig. 1. Conceptual seven-layer architecture for objective-aware rollback.

Objective uncertainty (Layer 1) is important because treating an inferred instruction as perfectly known can create incentives to pursue an incorrect interpretation [Hadfield-Menell et al. 2017]. Every external action receives a unique provenance identifier (Layer 2), $a_i = (id_i, o_i, p_i, e_i, r_i)$, enabling the controller to distinguish fully reversible, compensatable, irreversible, and status-unknown actions. Checkpoints (Layer 3) are created at semantic boundaries rather than fixed intervals. The drift monitor (Layer 4) compares behavior against the validated objective using multiple signals rather than a single reward channel, since reward hacking is itself one of the failure modes being detected. The interrupt controller (Layer 5) must sit outside the agent’s own authority,

following the corrigibility logic of Soares et al. [2015] and Orseau and Armstrong [2016]. State restoration (Layer 6) must cover model state, filesystem state, credentials, and task metadata, since agent-sandbox research shows relevant state can exist at the operating-system level even when absent from the conversation history [Wu et al. 2026]. Finally, post-rollback validation (Layer 7) prevents the system from restoring an earlier state and immediately reproducing the same objective error.

7 Research Challenges and Limitations

Several open challenges limit the practical deployment of rollback as a safety mechanism.

Detecting objective misinterpretation is difficult because a capable agent may produce behavior that appears locally reasonable while being globally wrong; goal misgeneralization shows competence can remain high even when the objective is incorrect [Di Langosco et al. 2022].

The irreversibility problem is the most fundamental limitation: many real-world actions cannot be perfectly reversed. Rollback should therefore distinguish *reversible*, *recoverable*, and *compensatable* effects, since a system that claims “rollback safety” without modeling this distinction may provide false assurance.

Hidden and external state is distributed across model memory, operating-system processes, caches, external APIs, and third-party services, so a local snapshot cannot guarantee global restoration [Wu et al. 2026].

Rollback-induced nondeterminism means an LLM agent may generate different actions after restoration because sampling or hidden environment state has changed; $S_t \rightarrow A_t$ does not imply that restoring S_t reproduces A_t , motivating semantic action logs and effect-aware replay mechanisms [Zheng et al. 2026].

Memory and objective persistence raises a causal-attribution problem: if an agent has already written an erroneous interpretation into persistent memory, restoring the filesystem without addressing that memory may cause the same failure to recur.

Adversarial agents make the independence of the rollback controller essential, since reward-tampering research shows models can, under some training conditions, generalize toward manipulating their own evaluation mechanism [Denison et al. 2024].

8 Research Gaps and Future Directions

Building on the comparative analysis in Section 5 and the challenges in Section 7, this review identifies the following concrete research gaps:

- (1) **Objective-aware checkpointing**: current systems (DeltaBox, Crab) optimize computational recovery only; checkpoints should carry an explicit *objective certificate* $\Gamma_t = (S_t, O_t, V_t, P_t)$ recording validation evidence and permission state.
- (2) **Standardized rollback benchmarks**: no existing benchmark measures objective-drift detection latency, rollback success rate, or residual state after restoration in a unified way.
- (3) **Irreversible-action protocols**: high-impact tool calls need transactional semantics (*prepare* $\rightarrow$ *validate* $\rightarrow$ *commit* $\rightarrow$ *observe*) so that authorization is checked before an action becomes irreversible.
- (4) **Semantic idempotency**: external actions should be bound to unique execution and checkpoint identifiers so a server can distinguish a legitimate retry from a post-rollback replay, directly addressing the attack class identified by Zheng et al. [2026].
- (5) **Objective version control**: objectives should be versioned ($O_1 \rightarrow O_2 \rightarrow O_3$) rather than silently overwritten when a user clarifies an instruction.

- (6) **Formal verification of recovery:** invariants such as $\text{Resume}(S_t) \Rightarrow \text{Validated}(O_t) \wedge \text{ValidPermissions}(T_t)$ remain largely unexplored for LLM-based agents specifically.
- (7) **Human-in-the-loop escalation:** a practical five-level hierarchy (continue; pause and ask; auto-rollback; revoke and require approval; isolate/shutdown) needs empirical validation on real deployed agents, not only conceptual specification.

The most significant gap, spanning all seven items above, is the integration of AI-alignment concepts with distributed-systems recovery semantics: moving from checkpointing *state* toward checkpointing *validated intent and authority*.

9 Discussion

Alignment methods attempt to make the agent pursue the intended objective; monitoring attempts to detect divergence; corrigibility attempts to preserve human authority; rollback attempts to contain the consequences of divergence after it has begun. This review’s comparative analysis (Table 3) shows these are complementary layers rather than substitutes: reward hacking arises because proxies diverge from intended objectives [Skalse et al. 2022]; goal misgeneralization shows correct training rewards do not guarantee correct objectives under distribution shift [Di Langosco et al. 2022]; and recent systems research demonstrates that technically efficient checkpoint/restore is possible while simultaneously revealing that naive restoration can itself introduce new hazards [Dong et al. 2026; Wu et al. 2026; Zheng et al. 2026]. The resulting design principle is that an agent should never be considered safely recoverable merely because its computational state can be restored — it should be considered recoverable only when its objective state, authority state, and external effects can also be brought back into a validated configuration.

10 Conclusion

This review examined rollback as a complementary AI-safety mechanism for objective misinterpretation in agentic systems. By synthesizing the literature on specification gaming, reward hacking, goal misgeneralization, mesa-optimization, corrigibility, and safe interruption, and by directly comparing leading checkpoint/restore and reward-hacking-mitigation systems (Table 3), this paper argued that rollback should be treated as a transactional safety primitive within a layered architecture rather than a standalone alignment solution. The most significant open research direction remains the integration of AI-alignment concepts with distributed-systems recovery semantics: an agent execution model analogous to a safety-critical transaction, in which actions are proposed, independently evaluated, committed only under appropriate authorization, and recoverable when evidence indicates the agent’s interpretation of the task has diverged from human intent.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016). <https://arxiv.org/abs/1606.06565>.
- Ö. V. Çağatan and Xin Zhao. 2026. Reward Hacking in Language Model Agents: Revisiting AI Safety Gridworlds. *arXiv preprint arXiv:2606.15385* (2026). <https://arxiv.org/abs/2606.15385>.
- Carson Denison, Michael MacDiarmid, Felix Barez, David Duvenaud, Sophie Kravec, Samuel Marks, Nick Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. 2024. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. *arXiv preprint arXiv:2406.10162* (2024). <https://arxiv.org/abs/2406.10162>.
- Lauro L. Di Langosco, Jack Koch, Liam D. Sharkey, Jacob Pfau, and David Krueger. 2022. Goal Misgeneralization in Deep Reinforcement Learning. In *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162. PMLR, 12004–12019. <https://proceedings.mlr.press/v162/langosco22a.html>; arXiv:2105.14111.
- Y. Dong, J. He, Y. Hou, D. Du, Z. Xu, S. Yu, Y. Xia, and H. Chen. 2026. DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback. *arXiv preprint arXiv:2605.22781* (2026). DOI-style identifier: 10.48550/arXiv.2605.22781; <https://arxiv.org/abs/2605.22781>.

- Sebastian Farquhar, Vikrant Varma, David Lindner, Dylan Elson, Charlie Biddulph, Ian Goodfellow, and Rohin Shah. 2025. MONA: Myopic Optimization with Non-myopic Approval Can Mitigate Multi-step Reward Hacking. *arXiv preprint arXiv:2501.13011* (2025). PMLR 267; <https://proceedings.mlr.press/v267/farquhar25a.html>.
- Dylan Hadfield-Menell, Anca D. Dragan, Pieter Abbeel, and Stuart Russell. 2017. The Off-Switch Game. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)*. 220–227. DOI: 10.24963/IJCAI.2017/32; <https://doi.org/10.24963/IJCAI.2017/32>.
- Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Samuel Garrabrant. 2019. Risks from Learned Optimization in Advanced Machine Learning Systems. *arXiv preprint arXiv:1906.01820* (2019). <https://arxiv.org/abs/1906.01820>.
- Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Rishabh Kumar, Zachary Kenton, Jan Leike, and Shane Legg. 2020. Specification Gaming: The Flip Side of AI Ingenuity. Google DeepMind Research. <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>.
- Laurent Orseau and Stuart Armstrong. 2016. Safely Interruptible Agents. In *Proceedings of the 32nd Conference on Uncertainty in Artificial Intelligence (UAI 2016)*. AUAI Press, 557–566. ISBN: 978-0-9966431-1-5; ACM record: <https://dl.acm.org/doi/10.5555/3020948.3021006>.
- S. Reddy, Anca D. Dragan, Sergey Levine, Shane Legg, and Jan Leike. 2019. Learning Human Objectives by Evaluating Hypothetical Behavior (ReQueST). *arXiv preprint arXiv:1912.05652* (2019). <https://arxiv.org/abs/1912.05652>.
- Joar M. V. Skalse, Nathan H. Howe, Victoria Krakovna, and David Krueger. 2022. Defining and Characterizing Reward Gaming. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*. OpenReview: <https://openreview.net/forum?id=yb3HOXO3IX2>.
- Nate Soares, Benja Fallenstein, Eliezer Yudkowsky, and Stuart Armstrong. 2015. *Corrigibility*. Technical Report. Machine Intelligence Research Institute / Future of Humanity Institute.
- Elliot Thornley. 2024. The Shutdown Problem: An AI Engineering Puzzle for Decision Theorists. *arXiv preprint arXiv:2403.04471* (2024). <https://arxiv.org/abs/2403.04471>.
- Alexander M. Turner, Logan Smith, Rohin Shah, Andrew Critch, and Prasad Tadepalli. 2021. Optimal Policies Tend to Seek Power. *arXiv preprint arXiv:1912.01683* (2021). <https://arxiv.org/abs/1912.01683>.
- T. Wu, C. Chang, L. Cao, W. Gao, and W. Wang. 2026. Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes. *arXiv preprint arXiv:2604.28138* (2026). DOI-style identifier: 10.48550/arXiv.2604.28138; <https://arxiv.org/abs/2604.28138>.
- Y. Zheng, Y. Yang, W. Zhang, and A. Quinn. 2026. ACRFence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore. *arXiv preprint arXiv:2603.20625* (2026). DOI-style identifier: 10.48550/arXiv.2603.20625; <https://arxiv.org/abs/2603.20625>.